

Control variate method

```
MCcontrol<-function(f,g,m,r1,r2,control=TRUE){
  mu<-integrate(f,lower=r1,upper=r2)$value
  u<-runif(m,r1,r2)
  B<-f(u)
  A<-g(u)
  corr<-cor(A,B)
  a<-cov(A,B)/var(B)
  if(!control)
    T1<-g(u) else T1<-g(u)+a*(B-mu)
  M<-(r2-r1)*mean(T1)
  if(!control)
    list(MC=M) else list(Correlation =corr, MC=M)
}
```

1. Using Monte-Carlo integration with control variate, approximate the integral

Compare this result with the result obtained by simple Monte-Carlo integration.

2. Use method of control variates to estimate . Note that for and .

(a) Use control variate , (b) Use control variate .

(c) Compare both (a) and (b) with simple Monte Carlo method. Which one achieves greater variance reduction?

3. Approximate the integration of 1 and 2 using MC method with control variates and regression.

4. Using the linear congruential method for random number generation, generate the first 3 random numbers using a seed of 2985 and parameters and . Reports random numbers to 5 decimal places.

5. For 32 bit computers, combine generators with , and , . Consider seed = 73.

```
ran<-function(m,a,seed, n){ # m and a are vectors of length k
  k<-length(m)
  start<-rep(seed,k)
  vec<-c()
  for(i in 1:n){
    z<-rep(0,k)
    sum<-0
    for(j in 1:k){
      z[j]<-(a[j]*start[j])%%m[j]
      start[j]<-z[j]
      sum<-sum+((-1)^(j-1))*z[j]
    }
    x<-sum%%(m[1]-1)
    if(x>0)
```

```

        R<-(x/m[1]) else (m[1]-1)/m[1]
        vec<-c(vec,R)
    }
    vec
}
m<-c(2147483563,2147483399); a<-c(40014,40692); seed<-73; n<-10
ran(m,a,seed,n)

```

6. (a) Random sample generation from exponential distribution with rate [exponential()] or

mean .

Write r-code to draw a random sample of size 1000 from exponential distribution with rate 2. Show that mean and variance of the sample is close agreement with 0.5 and 0.25 respectively.

Draw density curve and histogram using the sample. Density curve on histogram

Test the hypothesis that

```

rexpon<-function(size,lambda){      #size = sample size, lambda = parameter of the
    X<-rep(0,size)                  # distribution
    for(i in 1:size){
        U<-runif(1)
        X[i]<--(1/lambda)*(log(U))
    }
    X
}
set.seed(100)
x<- rexpon(10000,2)
library(MASS)
win.graph(6,3,8)
par(mfrow=c(1,2))
par(mar=c(4,4,1,1))
plot(density(x))
truehist(x)
# to draw density curve on histogram, the code : lines(density(x))
# test statistic
z<- (mean(x)-2)/(sd(x)/sqrt(10000)) ## z ~ n(0,1)

```

(b) Gamma () random variable generation using uniform random variate (shape parameter and rate parameter

```

rgam<-function(size, shape, rate){      # size = sample size, shape =shape
    x<-rep(0,size)                      # parameter, rate = rate parameter
    for ( i in 1:size){

```

```

        x[i]<-(-1/rate)*sum(log(runif(shape)))
      }
    }
  }
}

```

```

set.seed(100)
x<- rgam(10000,3,2)
library(MASS)
win.graph(6,3,8)
par(mfrow=c(1,2))
par(mar=c(4,4,1,1))
plot(density(x))
truehist(x)
# test the hypothesis that
# z<- (mean(x)-(3/2))/sd(x)/(sqrt(10000))

```

(c) **Box-Muller algorithm:** R code to generate a random sample of size 10000 from a normal distribution with mean 5 and standard deviation 2 using Box-Muller algorithm.

```

rnormal<-function(size, Mean, Sd){
  L<-size/2
  x<-rep(0,size)
  for(i in 1:L){
    u1<-runif(1)
    u2<-runif(1)
    x[2*i-1]<-Mean+Sd*sqrt(-2*log(u1))*cos(2*pi*u2)
    x[2*i]<-Mean+Sd*sqrt(-2*log(u1))*sin(2*pi*u2)
  }
  x
}

set.seed(100)
x<- rnormal(10000,5,2)
library(MASS)
win.graph(6,2.5,8)
par(mfrow=c(1,2))
par(mar=c(4,4,1,1))
plot(density(x))
truehist(x)
# test the hypothesis that
z<- (mean(x)-5)/(sd(x)/sqrt(10000)) # z~N(0,1)

```

```
# test the hypothesis that
chi<-(10000-1)*(sd(x))^2/4
qchisq(.01,10000-1,lower.tail=FALSE)
pval<-pchisq(chi,10000-1,lower.tail=FALSE)
```

(d) Chi-square variate generation: Write R-code to generate random sample of size 10000 from chi-square (10) distribution. Plot density curve, histogram, and density curve on histogram

```
rchisquare<-function(size, df){
  x<-rep(0,size)
  for ( i in 1:size){
    x[i]<-(-2)*sum(log(runif(df/2)))
  }
  x
}

set.seed(100)
x<- rchisquare (10000,10)
library(MASS)
win.graph(6,2.5,8)
par(mfrow=c(1,2))
par(mar=c(4,4,1,1))
plot(density(x))
truehist(x)
lines (density(x))
```

(e) Random sample from binomial distribution

Write r-code to simulate a random sample of size 1000 from binomial (7, 0.6) distribution. Draw line diagram and barplot.

Test the hypothesis that $H_0: p=0.6$

```
rbino<-function(size,n,p){  # size = sample size, n = number of trials,
  X<-rep(0, size)           # p = probability of success
  for(i in 1:size){
    x<-0
    sum<-choose(n,x)*(p^x)*(1-p)^(n-x)
    u<-runif(1)
    while(u>sum){
      x<-x+1
      sum<-sum+choose(n,x)*(p^x)*(1-p)^(n-x)
    }
    X[i]<-x
  }
  X
```

```

    }
set.seed(100)
n<-7
p<-0.6
N<-10000
x<- rbino(N,n,p)
win.graph(6,2.5,8)
par(mfrow=c(1,2))
par(mar=c(4,4,1,1))
plot(density(x))
freq<-table(x)
barplot(freq)
plot(freq)
mu<-n*p
phat<-mean(x)/n
z<-(mean(x)-mu)/sqrt(n*phat*(1-phat)/N)
#or
phat<- mean(x)/n
z<-(mean(x)/n-p)/sqrt(phat*(1-phat)/(n*N))

```

(f) Random sample from Poisson distribution

Write r-code to simulate a random sample of size 10000 from Poisson (2) distribution

Test the hypothesis that H_0 :

```

rPoisson<-function(size,lambda){  # size = sample size, lambda = Poisson
  y<-rep(0, size)                # parameter
  for(i in 1:size){
    x<-0
    sum<-(exp(-lambda)*(lambda)^x)/factorial(x)
    u<-runif(1)
    while(u>sum){
      x<-x+1
      sum<-sum+(exp(-lambda)*(lambda)^x)/factorial(x)
    }
    y[i]<-x
  }
  y
}

set.seed(100)
N<-10000
lambda<-2

```

```

x<- rPoisson (N, lambda)
win.graph(6,2.5,8)
par(mfrow=c(1,2))
par(mar=c(4,4,1,1))
freq<-table(x)
barplot(freq)
plot(freq,type="h")
lambhat<-mean(x)
z<- ( lambhat-lambda)/sqrt(lambhat/N)

```

(g) Geometric random variable generation using uniform (0,1)

Write r-code to simulate a random sample of size 10000 from geometric (0.25) distribution

```

rgeometric<-function(size,prob){  # size = sample size, prob = parameter of the
  x<-rep(0,size)                  # distribution
  for(i in 1:size){
    u<-runif(1,0,1)
    x[i]<-floor(log(u)/log(1-prob))
  }
  x
}
set.seed(100)
x<- rgeometric (10000,0.25)
win.graph(6,2.5,8)
par(mfrow=c(1,2))
par(mar=c(4,4,1,1))
freq<-table(x)
barplot(freq)
plot(freq,type="h")

```

(h) Write R-code, using Marsaglia's polar algorithm to generate a random sample of size 10000 from normal(3, 4) random variables.

#R-code

```

rmarsag<-function(size, Mean, Sd){
  x<-rep(0,size)
  L<-size/2
  for(i in 1:L){
    n<-1
    while(n==1){
      u1<-runif(1)

```

```

        u2<-runif(1)
        w1<-(2*u1)-1
        w2<-(2*u2)-1
        w<-w1^2+w2^2
        if(w<1){
            x[2*i-1]<-Mean+Sd*sqrt(-2*log(w)/w)*(w1)
            x[2*i]<-Mean+Sd*sqrt(-2*log(w)/w)*(w2)
            n<-0
        }
    }
    x
}

set.seed(100)
x<- rmarsag (10000,3,4)
library(MASS)
win.graph(6,2.5,8)
par(mfrow=c(1,2))
par(mar=c(4,4,1,1))
plot(density(x))
truehist(x)
lines (density(x)

```